

Building Meta's Threads App (Real-World Engineering Challenges)

The Threads app was downloaded by more than 100M people on launch week. How did the engineering team build the app, and handle an unexpectedly intense launch? Exclusive.



GERGELY OROSZ

SEP 7, 2023 • PAID



36



3

Share

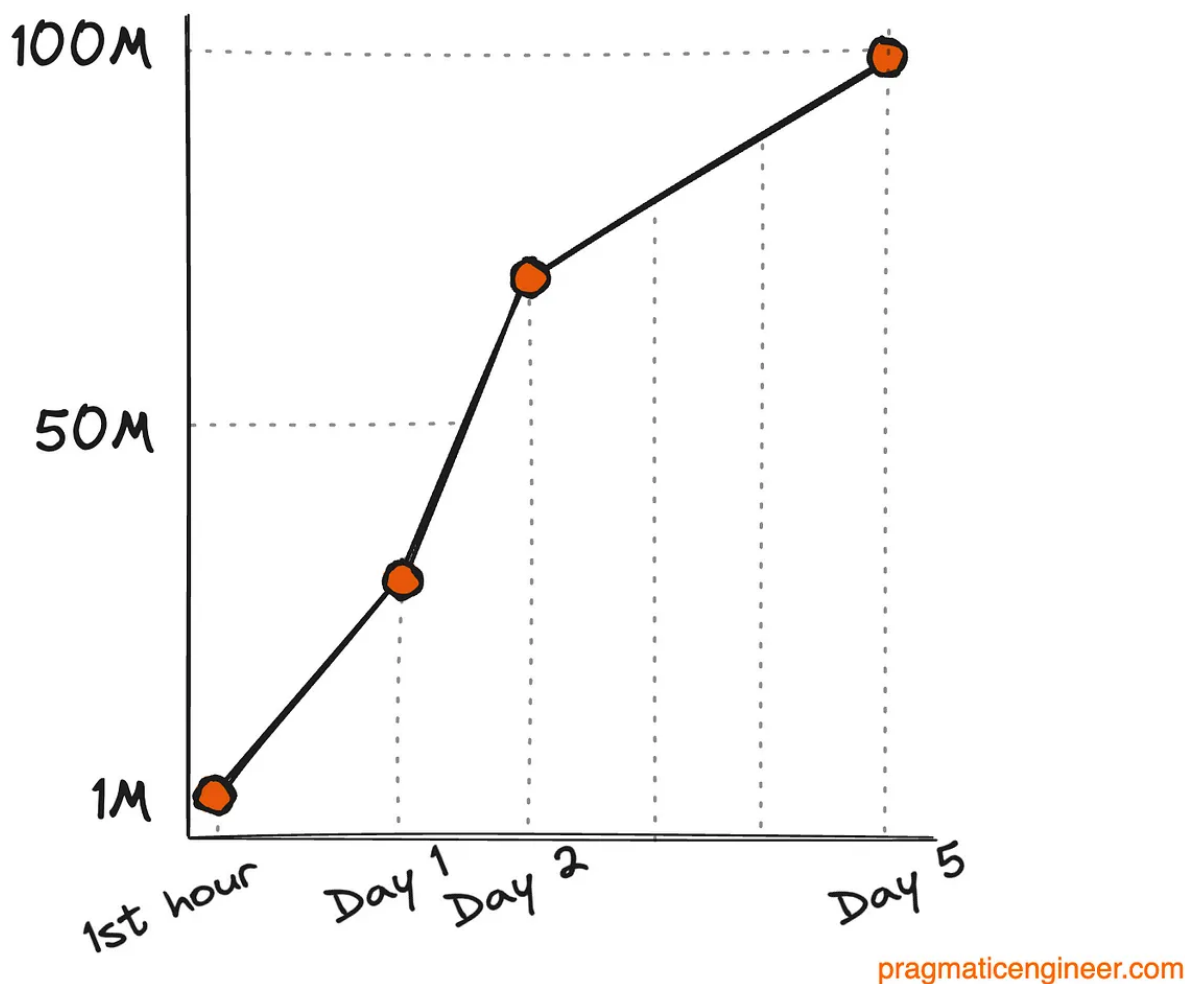


'Real-world engineering challenges' is [a series](#) in which I interpret interesting software engineering or engineering management case studies from tech companies. In today's issue, we peek behind the scenes of the a major app launch at Meta: that of [Threads](#).

If you're not a subscriber to the newsletter yet, you missed the issue on [Why cloud development environments are spiking in popularity right now](#) and [a few others](#).
Subscribe to get issues like this every week. 📌

In July of this year, Meta launched its latest mobile app, Threads, a microblogging service and new rival to X, formerly Twitter. In the first five days following its launch, the app achieved 100M downloads – a new record for the company by some margin. Meta's previous record for new app installs in the first 5 days after launch was 1M. Here is how the growth curve looked like, for the number of onboarded users:

Number of new users on Threads, on launch week



The growth of the Threads app in the first week. As Threads was mobile-app-only on launch, this number also equates to the number of app installs

These are huge numbers for a new app, even when taking into account that the more than 2 billion monthly active Instagram users were promoted to install this new app. When I learned of these stats, my first thought was wondering how did the engineering team pull off this launch, seemingly without any major hiccups? Especially, when considering that the Threads team moved up the launch, after Twitter [limited free users to viewing 600 posts a day](#): and this decision created the perfect launch opportunity for a Twitter challenger. Threads launched within a few days after Twitter's rate limiting: and it seems like the app seized that moment well.

But what was it like, from the inside, for engineers building the app? For answers, I reached out to the Threads engineering team, who were open to sharing behind-the-scenes details. [Jesse Chen](#) (engineering manager on Threads) and [Zahan Malkani](#) (server engineer lead on the app) told me the story on how Threads was built. *The details they shared fit nicely into the [Real-World engineering challenges](#) series.*

In today's issue, we dive into:

1. Building Threads
2. Technology choices and engineering approaches
3. Planning for the launch
4. The launch
5. Learnings and next steps

In this article, my questions are in italic and answers from the Threads engineering team are in normal font. With that, it's over to Jesse and Zahan, to lift the lid on how the app was built.

1. Building Threads

How "typical" was Threads as a project, compared to other app launches?

Threads was different from most launches. Usually when there is a new, exciting top-down priority, we lean into our open internal culture to rally the company around the initiative and get folks excited to join and bootstrap the team.

In this case, however, we kept the team deliberately small and confidential. We tapped the shoulders of some of our strongest engineers, product managers, and designers within Instagram to work on this, and had regular reviews with Adam Mosseri, the head of Instagram. We knew time to market was a priority, and wanted to shield the team from noise.

The launch team

How large was the initial team, and what portion of it was engineering?

The team was lean, flat and engineering-heavy, which was an intentional organizational design choice. We had a tight deadline and we wanted to maximize quick decision making and minimize bureaucracy.

We believed a leaner organization could help us build higher quality products faster, and this proved correct. We started with a very small team, with around a dozen people in total. The vast majority of the whole team came from Instagram. Everyone joining the Threads building towards launch was an internal transfer: so they were working at Meta beforehand..

The peak staffing on the team was a week before launch. At this time the team consisted of:

- 3 product managers
- 3 designers
- About 60 engineers

I (Jesse) worked with engineering leads across the Instagram org to identify key senior engineers to pull onto this project. The experience level of the team skewed senior, since we had a relatively small team and needed engineers who could operate autonomously, and own their respective pieces of the app with little oversight needed.



The Threads team at a launch happy hour celebration in July 2023, at San Francisco, California

Planning and working with the rest of Meta

How formal or informal was the planning, versus jumping straight to prototyping/coding?

There was some high-level planning in the beginning, as we debated how much to reuse Instagram's codebase. However, for the most part, we jumped straight into prototyping and coding.

We evoked one of Meta's early mantras, "code wins arguments." We aligned heavily with this principle. Even before we received the official greenlight, engineers were already prototyping the idea of a text feed, to prove feasibility!

How much did you work with/rely on other internal teams within Meta – such as infra, platform, or product teams?

One unique collaboration we had was a "Bug Bounty" program where we invited security researchers to probe the app for bugs before launch. The goal of this program was to help protect people's data, and uncover any security or privacy bugs we might have early on. We discovered a few notable bugs thanks to this program.

We worked in secrecy. Pre-launch, the core engineering team was isolated from the rest of Instagram's engineering org. Working in secrecy, we added our code to the shared codebase discreetly and only looped in partner teams when we faced issues which required their expertise.

Still, we could not have launched this app in the time it took by working in isolation. We relied heavily on our internal tooling and frameworks. We leveraged several things from other internal teams, like:

- UI frameworks
- APIs
- Login, privacy, security, integrity infrastructure
- Server infrastructure
- ... and much more!

We reused as much code as we could to move as quickly as possible. Our speedy development speed and stability during launch is a testimony of how awesome our engineering organization is. Our platform/infra teams make it look easy when we quickly scale and build new products, standing on their shoulders.

Timeline

We wrote the first line of code for the standalone app late January 2023. We submitted the final launch candidate to the app store in June 2023.

The engineering effort from start to launch took 5 months.

2. Technology choices and engineering approaches

Which technologies did you choose for the backend and the mobile apps?

The backend is running on Instagram's stack – our custom **Python** running **Django** speaking to a custom REST API. We also talk to Meta's bigger monolithic backend over **GraphQL** which is on [HHVM](#) – a virtual machine for executing programs written in the Hack, PHP-like language.

The apps are largely native, using a mix of **Swift** on iOS (and a little Objective C), and **Jetpack Compose** on Android (with Kotlin and Java as languages). There are some shared server-rendered screens for some simple experiences, but native is the norm.

Reusing the internal stack

Which choices did you make in reusing existing parts of the stack – like libraries and frameworks – versus building your own?

Because time-to-market was a priority, we used the Instagram tech stack wherever possible. Instagram itself is built upon a variety of libraries and frameworks, many of them in house.

We used the latitude afforded by a greenfield opportunity to try out technologies at scale that we hadn't had the chance to, before. On iOS, all new parts of the app were built with Swift, and on Android we used [Jetpack Compose](#) to a large degree. We credit the new language (Swift) and framework choices (Jetpack Compose) with helping us move as quickly as we did.

We chose to heavily leverage Instagram's existing tech stack, which is built upon Meta's infrastructure, which contributed greatly to our dev speed. This choice took the ambiguous scope problem from:

"How do we create a new text-based social network from scratch?"

To this much more focused question:

"How do we use the building blocks of the Instagram feed for posts that are more text-based?"

Threads felt snappy on launch. How much did you prioritize client-side performance, and how did you measure this or test for it, if you did?

Launch performance was not snappy enough! We have internal tools which profile and understand where time is spent, and we had to employ them extensively at launch, when the most popular posts individually started getting the kind of engagement we didn't expect the entire network would get on launch day.

We have internal frameworks which let us measure a variety of crucial performance metrics. Here are some examples:

- "Time to new content" is for cold start to seeing content in feed
- Scroll performance is checked through looking for frame drops

The testing approach for Threads

How did you go about (or skip) testing; specifically automated tests? There are two schools of thought here:

- A) build it and get rapid feedback. Worry about automated tests when there's a product-market fit (PMF.)
- B) build it from scratch assuming a PMF, and put tests in place for maintainability, etc.

We were in camp A, of worrying about automated tests after PMF for frontend / UI tests. Because of how important time to market is, we were disciplined about dropping things which didn't help us towards that goal.

To be sure, automated tests are important; however, there is no value in writing tests for a UI that is constantly changing. As context, some flows had 3+ redesigns before we landed on something we liked!

Automated tests don't help a product with no product market fit. Instead, we relied extensively on dogfooding and QA. Dogfooders downloaded master builds that updated every few hours with the latest changes. The quick feedback loop allowed us to go from figma mockups, to a master build deployed onto your phone, within a few hours.

That said, backend business logic followed a test-driven development (TDD) practice that was encouraged through peer review. We have great internal testing frameworks that let you set up state and run through actual business logic fairly easily, without touching production databases. This enabled the backend team to move quicker, since we could make changes with confidence, without having to end-to-end test each change.

3. Planning for the launch

What were the biggest load challenges you anticipated for Threads?

A particular feature we knew was going to create scaling challenges was giving users the ability to pull across their graph from Instagram.

One of the difficult parts of starting up a new social network is the "cold start" users face. Every one of your new users needs to find the people they're interested in on the platform and choose to follow / friend them. In our case, we wanted to make that easier by offering a one tap action to "follow everyone you already follow on Instagram."

There is a major scaling issue here, though! Think of the more popular accounts joining Threads, with audiences in the order of millions already queued up to follow them. We needed to process such large accounts in a timely manner.

After launch, we realized that the scale we were dealing with *far* exceeded our expectations. Since the launch, we've needed to redesign this system with a totally different set of assumptions.

Dogfooding

For how long was Threads dogfooded, and by roughly how many folks? How did the team go about collecting feedback and addressing it?

The app was in constant dogfooding from the start, but we intentionally kept the feedback group small. Our goal was to get high signal feedback without overwhelming the team with bug reports.

We slowly expanded dogfooding from the core team, to all of Instagram. With the smaller dogfooding population, engineers engaged directly and debugged with coworkers, resulting in a really tight feedback loop.

(Not) load testing

How did you prepare for the launch, and to handle the initial load? For example, did you do load tests in advance, or other testing ahead of time?

Doing extensive tests was certainly the plan of record. However, due to external circumstances, we pulled our launch to earlier than planned.

We had to scale our capacity and support the launch, on the fly. This was due to an earlier than planned launch, and because of this timeline change, we were unable to do load tests beforehand. This was one of the most intense launches we've had at the company in a while!

How certain or uncertain were you that you could handle a potentially large load on launch? I found it unusual to do a global launch with no "signup limiting" or waitlist in place, as you did.

We knew we had the expertise of Instagram and Meta behind us, so we were fairly confident for the launch. And we initially thought we'd be a small drop in the bucket at Meta's scale.

The reality of onboarding so many users, so quickly, was unprecedented at Meta, in hindsight. However, the decade of investment in reliable and performant infra at Meta helped. Investment areas like:

- Databases

- Graph indexing systems
- Ranking systems
- Integrity systems
- Notification systems

These systems all worked in concert to soak up the load without any major hitches.

4. The launch

From the outside, the launch looked surprisingly smooth, with perhaps a few brief blips during the first week. From the inside, how did it go?

This was an “all hands on deck” situation for the team, starting the week before launch, and for the weeks afterwards. We hosted regular video calls to talk through issues live, and a majority of the team was working around the clock to resolve various issues.

The heartening thing was that it was decidedly not a “Threads team-only” effort. A major part of launch support came from partner teams in Instagram, and across Meta. Devs working on infra components collaborated with production engineers keeping the systems up, who worked with product engineers debugging their features.

A launch like this was one where you see the benefits of an open culture. An open codebase really pays dividends, as new people could quickly ramp up and provide value, even if it's a new domain.

Measuring how the launch went

Which engineering KPIs or measurements did you focus on, to ensure Threads worked as expected?

We used monitoring tools like:

- [ODS](#) – a high-performance time series storage engine – and [Scuba](#) – a data management system we use for real-time analysis. These tools help keep track of

load and the like in real-time.

- An evolved warehouse extract-transform (ETL) ecosystem to give us key product metrics hourly.

We heavily used PREQ (performance, reliability, efficiency, quality) tools. Our team leaned on the tremendous experience at Meta of supporting PREQ tools that were built into the client app, and integrated in our server-side frameworks.

Metrics we paid attention to included:

- Performance and server-side load
- "Time to new content" – our cold start app metric
- QPS / success rate on our key endpoints
- Queue length on async jobs handling heavy lifting
- Our general footprint on the main Instagram backend monolith

Endpoint-level catch-all alarms also notify us when things started to go south for particular endpoints, if we hadn't accounted for scale properly.

Launch stats

The previous record for the fastest-ever growing app at Meta was 1M users in 5 days. This is how Threads grew:

- 1M users: in 1 hour
- 30M: 1 day
- 70M: 2 days
- 100M: 5 days

5. Learnings and next steps

What was a difficult challenge which arose?

Making an engaging feed for Threads was the biggest one. On a microblogging site, the thing that is most relevant is what is happening *right now*. It needs to capture what everyone is talking about, and present choices for which conversations to dive into.

There's a balance between immediacy and helping the user find content from someone they're most likely to engage with. At the same time, other apps in the space have proved you don't need an extensive graph of connections in order to serve relevant and interesting content.

We've talked on the Threads team about how we'd like to reduce our reliance on users manually curating follow graphs. However, to reduce reliance on manual curation, you need to understand what these posts are about, and know what is going on in the world – which are both challenging, to say the least!

Learnings for the engineering team

What went well, from an engineering point of view in this project and launch?

These two stand out:

- The speed at which we were able to build a polished product for launch
- The relatively small size of the team which achieved this was viewed favorably both internally and externally.

There was a lot of market interest in this app before it launched, which gave people a good reason to download the app and try it out. Our job as engineers was to get roadblocks out of the way and let them do that. It's a huge accomplishment that we were able to do so relatively smoothly.

What valuable learnings are there for the engineering team?

Focus: it's the key differentiator between products set up to find product-market-fit and those which are not. Focus is necessary, although not sufficient by itself. The effort in Threads was to be able to focus by aggressively descoping the product experience to just the basics. Early on, we decided to focus execution on internal milestones.

Each internal milestone meant a fully polished, and *shippable* app experience, but with reduced functionality built out. These internal milestones forced clarity on what we *absolutely* needed to build, since there was never time to look at anything else.

What was the most interesting part of this project, as an engineer?

Understanding the breadth of the Instagram codebase. The technical decision to share much of the tech stack with Instagram meant that building this app became an exercise in going through Instagram's codebase, so we could make targeted changes just to support our use case.

It's rare to get an excuse to understand Instagram at such a high level, while delving into implementation details on the critical Instagram subsystems! It was almost like a puzzle: we needed to figure out within which Instagram systems to make the most targeted changes that would yield the most dividend, while taking the lowest risk. This challenge ended up being the most interesting part of the project.

Next steps

What impact could the Threads launch have on how apps get built at Meta?

There have been many folks reaching out internally who are looking at Threads as a case study for future product launches. The sheer speed at which we shipped the first version of Threads is what sets this launch apart – with work on the app having only started earlier this year.

It is a notable cultural change to have a small but senior engineering team ship a standalone app with this sort of public reception in just five months, even at Meta. I'm excited to see other teams take inspiration and build faster, scrappier teams!

What are the next priorities for the team, and the growth plans?

For priorities, aside from building all the core product gaps our users are clamoring for, we are also heavily investing in significant infrastructure upgrades to make it more robust for Threads.

Our actual launch numbers far exceeded projections. Our leadership is optimistic about the trajectory of the app and as a result, we are investing in growing the engineering team.

The Threads started off initially as a small incubator team within the Instagram org. With the launch success, we are excited to see this grow into a more formal initiative.

What are things the engineering team is excited about, looking ahead?

We've been working on delivering new Threads features as quickly as possible. One such feature is the highly-requested logged-in web version of Threads, which we shipped last month. Another version is our expansion of the keyword content search test to more users in Canada, the United States, India, and the United Kingdom – which we [are launching today](#) (7th September).

I'm excited for our team to keep shipping features that will improve people's experience on Threads.

I'm also excited about our commitment to join the Fediverse and support the ActivityPub protocol. At our scale, it's an incredibly exciting technical and regulatory challenge. Joining the [Fediverse](#) and building interoperability with other networks to create a more diverse and thriving public conversation ecosystem is something we remain committed to working towards.

Takeaways

This is Gergely again.

Many thanks, [Jesse](#), [Zahan](#) and the Threads engineering team, for sharing details on how they built and launched the app. It was interesting to understand how the team pulled off this feat, and how their approaches stack up with what we know [about Meta's engineering culture](#) – a topic we cover in-depth, [in a two-part series](#).

One last question I had to ask the Threads team is the plans to launch in the European Union. The app is available globally – except for the EU, thanks to regulatory concerns. Here is what the team told me about their plans:

"We are working on launching Threads in more countries and will continue to evaluate whether to launch in Europe, but the upcoming regulatory uncertainty has played into our decision not to launch right now. Europe continues to be an incredibly important market for Meta and we hope to make new products available here in future."

It will be interesting to note if Meta delaying its EU launch is an exception: or if we can expect new apps and services launched by larger corporations to follow an "EU last" strategy, thanks to the additional privacy implications strict European regulations require.

What stood out to me from their accounts of the Threads project, is:

Moving as quickly as a well-run startup is happening inside Meta. Meta employs well over 50,000 people across hundreds of teams. Big Tech is often ridiculed for how slugging things become. And yet, within this environment the Threads team shipped a production-ready app in around 5 months!

There are several startups building Twitter-like apps: Bluesky, Nostr, and T2, are all examples. All are smaller in size – usually between 5-15 staff – but none went from zero to launch as quickly as the Threads team did.

Speed of execution used to be the advantage of startups over large corporations: this is partly how Facebook out-executed MySpace, around 20 years ago. That Meta managed to retain – or re-ignite! – this execution speed, as it is hundreds of times the size of a startup: this is remarkable.

Small, senior teams could be a way forward at companies like Meta. Threads seems to have been a roaring success inside the company. It sounds like the execution model could become a blueprint for greenfield projects: assemble a small and senior team, shield them from the larger organization, utilize existing platforms where possible, and launch quickly.

I can see other larger tech companies taking inspiration from this approach. But the challenge tends to not lie in assembling a small and senior team; it's to have a new

team not waiting on other internal dependencies. And for not waiting on other teams, Threads had a very significant sponsor: the head of Instagram himself, Adam Mosseri.

Having a senior-enough sponsor is critical for small teams to succeed in a large organization. My sense is that the Threads team could move as fast as they did and leverage Meta's internal infrastructure – as well as make changes to Instagram's services as needed – in great part thanks to support from the top.

Clearly, Threads was a priority within Instagram, and the biggest dependency Threads had on Meta's infrastructure was actually Instagram's infrastructure. It likely helped that many of the experienced engineers on Threads had worked on Instagram before, and were not new joiners, but had sufficient tenure at Meta.

Greenfield projects within a larger company are an excuse to try out technologies not yet widespread within an org. The downside of having successful products at any larger tech company is that the tech stack of those products starts to fall behind current trends. If you have a 'cash cow' product earning sizable revenue, then it's sensible to play it safe with technology choices, and to not rewrite the stack using newer languages or frameworks.

The nice thing about a greenfield project is that it's an excuse to onboard to stable-enough technologies which the rest of the company might not use as much. This is what the Threads team did by using more Swift – while many Meta teams work with Objective C – and utilizing Jetpack Compose.

This is an achievement that's hard enough for a startup to pull off, never mind a Big Tech giant.

Investing in infrastructure and platform teams pays off by soaking up unexpected loads. How did Threads withstand an unprecedented signup wave, with no major outages? This was down to Meta's infrastructure investments that created enough breathing space – capacity wise – to soak up load several times higher than the team forecasted.

The Threads team underestimated the high load the app would get, but thanks to building on Meta's infrastructure, the infra absorbed the additional load, and gave the

team valuable space until they rewrote parts of the system, to account for the additional scale.

Thanks for reading this article. I hope it was a revealing peek into how Meta pulled off the launch of Threads, a new rival to the platform formerly known as Twitter. It's refreshing to learn that just because an org is big, doesn't mean it cannot move fast.



36 Likes · 3 Restacks

Comments



Write a comment...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing